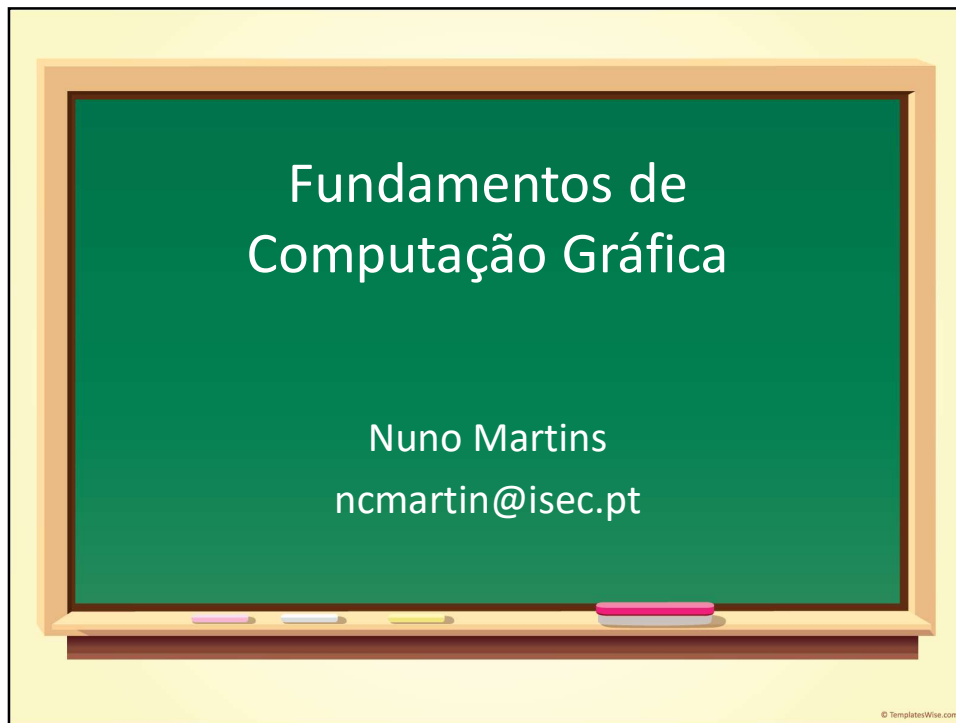




Técnicas discretas

- Durante muitos anos, a computação gráfica preocupou-se exclusivamente com primitivas para o desenho de objectos, não fornecendo ferramentas de acesso directo ao *frame buffer* ou a outros *buffers*.
 - Actualmente os API's permitem a escrita e leitura directa de pixels individuais nesses *buffers*.

Técnicas discretas

- Este avanço veio permitir o uso de algumas novas técnicas.
- As principais técnicas que fazem parte desse grupo e que interessam a esta unidade curricular são a:
 - do *anti-aliasing*;
 - do mapeamento de texturas;
 - da aplicação de nevoeiro;
 - operações sobre fragmentos.

Técnicas discretas

- Aliasing e anti-aliasing -

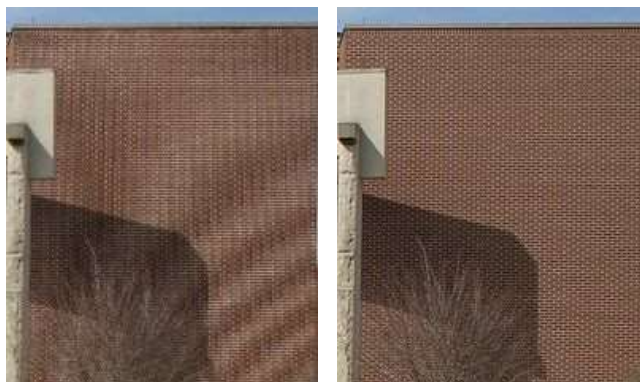


Figura 1. Qual a diferença entre as duas imagens?

Técnicas discretas

- *Aliasing* e *anti-aliasing* -

- Nas imagens em computador, por vezes, as linhas apresentam um aspecto em escada.
 - Defeito que pode ser visto até em ecrãs de resolução relativamente alta.
- Esse efeito é chamado de *aliasing espacial* e as técnicas usadas para o reduzir designam-se por *anti-aliasing*.
- Existe ainda o *aliasing* temporal.

Técnicas discretas

- *Aliasing* e *anti-aliasing* -

- O *aliasing* espacial é causado pelo facto da linha ideal ser aproximada por conjuntos de pixels localizados numa grelha discreta.
 - Isto é, acontece devido ao facto de se estar a representar algo cujo modelo matemático tem uma resolução infinita (porque é definido num espaço contínuo), numa representação visual com uma resolução limitada (por ser discreta).

Técnicas discretas

- Aliasing e anti-aliasing -

- Matematicamente as linhas são entidades unidimensionais que têm comprimento e não têm espessura.
- As linhas renderizadas num ecrã têm de ter espessura para poderem ser visíveis.

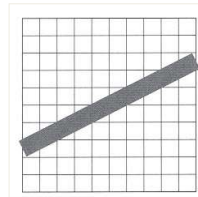


Figura 2. Linha ideal a renderizar.

Técnicas discretas

- Aliasing e anti-aliasing -

- Caso se desenhe a linha da figura 2, o que se faz por defeito em CG?
 - Definir uma espessura para a linha;
 - Assumindo uma espessura de um *pixel*, como a linha atravessa mais do que um *pixel* de cada vez, atribui-se a cada *pixel* por onde a linha passa o valor da respectiva cor.
 - Como se verifica que certos *pixels* não são ocupados na sua totalidade, a linha acaba por ser representada em escada.

Técnicas discretas

- Aliasing e anti-aliasing -

- Como se ultrapassa o problema de *aliasing* espacial criado aquando do desenho da linha da figura 2?
 - Usa-se, por exemplo, a técnica de *anti-aliasing* por média de área.
 - Que atribui a cada *pixel* por onde a linha passa uma percentagem da cor da linha que corresponda à percentagem de ocupação do *pixel* pela linha que lá passa.

Técnicas discretas

- Aliasing e anti-aliasing -

- Da aplicação desta técnica resulta uma imagem da linha com uma aparência mais alisada.

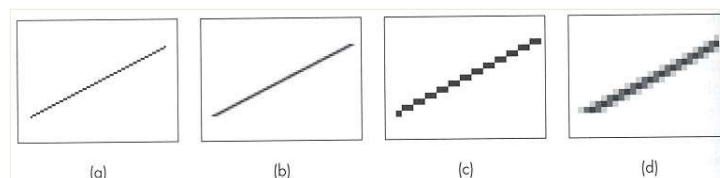


Figura 3. Linha ideal renderizada com *aliasing* (casos a e c) e com a aplicação da técnica de *anti-aliasing* (casos b e d).

Técnicas discretas

- Aliasing e anti-aliasing -

- O Blender permite lidar com o problema do *aliasing*.
 - Para isso tem que se usar o painel *Anti-Aliasing*, da janela de propriedades, quando o ícone *Render* estiver seleccionado.



Figura 4. Técnicas de *anti-aliasing* no Blender.

Técnicas discretas

- Texturas -

- O processo de modelizar um objecto por um conjunto de primitivas geométricas e fazer o seu rendering tem limitações.
- A representação de uma laranja, por exemplo, apesar de poder ter a forma e a cor certas, pode não ter um aspecto realístico que existe quando se olha para a casca rugosa da laranja.

Técnicas discretas

- Texturas -

- Assim, em vez de se tentar modelizar geometricamente a laranja de aspecto realista, de uma forma complexa, por meio de
 - um grande número de triângulos, cada um com propriedades próprias de modo a parecer uma laranja;
 - superfícies paramétricas.pode-se usar uma aproximação baseada no rendering.

Técnicas discretas

- Texturas -

- A aproximação baseada no rendering consiste em:
 - dividir a superfície em pequenas parcelas, ou fragmentos, e aplicar-lhes uma operação de mapeamento, de forma a lhes alterar certos atributos envolvido no processo de *shading*.
 - Este mapeamento pode ser feito por:
 - modificação do shading (sombreamento) com base num mapa (array bidimensional);
 - alteração dos parâmetros da superfície com base nesse mapa.

Técnicas discretas

- Texturas -

- Usualmente, a aproximação baseada em rendering pode ser vista na aplicação das seguintes técnicas:
 - Mapeamento de texturas,
 - que usa um padrão para alterar os coeficientes da reflexão difusa (cor), da reflexão especular e difusa (cor, brilho e refletividade –Environment mapping) e da transparência das superfícies 3D.
 - O padrão pode ser gerado através de uma imagem digitalizada ou via um algoritmo (método procedimental).
 - ...

Técnicas discretas

- Texturas -

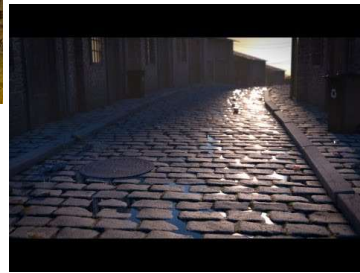
- (...) aplicação das seguintes técnicas:
 - ...
 - Mapeamento *Bump*,
 - Usa um padrão para distorcer a geometria da superfície (com base no vector normal) para criar variações locais;
 - A perturbação do vetor normal (*Bump Mapping*) ou a perturbação da superfície na direcção da normal (*Displacement Mapping*) é feita sem fazer nenhuma mudança real na forma do objeto (usa apenas a textura).

Técnicas discretas

- Texturas -



(a)



(b)

Figura 5. (a) Mapeamento de texturas. (b) Mapeamento *Bump*.

Técnicas discretas

- Texturas -

- As texturas e a geometria seguem caminhos separados no pipeline gráfico, encontrando-se na fase de renderização.

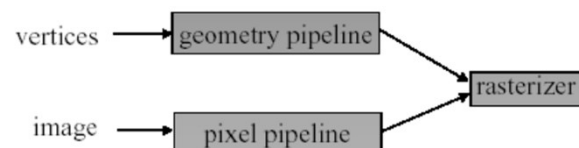
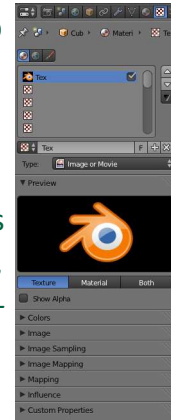


Figura 6. Pipelines de CG.

Técnicas discretas

- Geração de texturas -

- As texturas mais usadas na área da CG são as baseadas em imagens.
 - O padrão é igual ao conteúdo da imagem.
 - O Blender permite lidar com este tipo de texturas;
 - Tem que se usar vários painéis da janela de propriedades, quando o ícone *Textures* estiver seleccionado.



Técnicas discretas

- Geração de texturas -

- As texturas baseadas em imagens têm a vantagem de serem de fácil obtenção (basta tirar uma fotografia ou ir buscar à internet) e de demorarem pouco tempo a serem processadas (por isso, muito usadas nos jogos).
- As texturas baseadas em imagens têm a desvantagem de ocupar muito espaço.

Técnicas discretas

- Geração de texturas -

- Também podem existir texturas baseadas em algoritmos.
 - Designam-se por texturas procedimentais ou texturas 1D;
 - O padrão pode ser construído por um qualquer algoritmo;
 - Geram normalmente padrões semelhantes aos que se vê na natureza, como madeira, nuvens, mármore, etc.;
 - São normalmente constituídas por uma certa aleatoriedade.

Técnicas discretas

- Geração de texturas -

- Por exemplo, uma aproximação para criar o padrão de uma textura procedimental consiste em utilizar um gerador de ruído, seguido da aplicação de um filtro, como se pode ver na imagem 7.



Figura 7. Geração de uma textura procedimental.

Técnicas discretas

- Geração de texturas -

- Um exemplo de como se gera uma textura deste tipo encontra-se em:
https://en.wikibooks.org/wiki/Blender_3_D:_Noob_to_Pro/Procedural_Eyeball.
- Existem uma série de geradores de ruído, desde o deslocamento para algumas cores até o uso de funções trigonométricas e fractais.
 - Alguns dos métodos de ruído mais conhecidos são o Perlin Noise, o Lattice Noise e o Gradient Noise.

Técnicas discretas

- Geração de texturas -

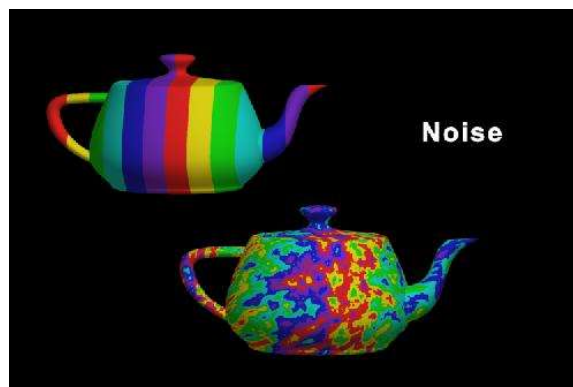


Figura 8. Aplicação de diferentes geradores de ruído na criação de texturas procedimentais.

Técnicas discretas

- Geração de texturas -

- As texturas procedimentais têm a vantagem de ocupar pouco espaço e de se poder facilmente fazer zoom in sem perda de qualidade.
 - Uma textura procedimental mal programada pode perder todas essas vantagens.
- As texturas procedimentais têm a desvantagens de demorarem mais a ser processadas.

Técnicas discretas

- Geração de texturas -

- O Blender também lida com texturas procedimentais.

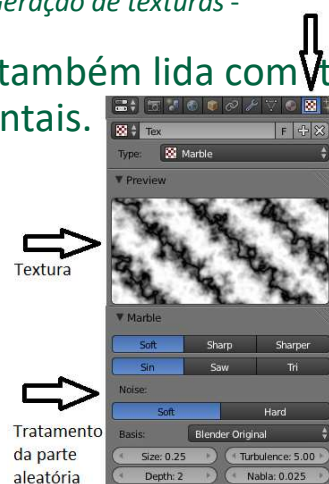


Figura 9. Textura procedimentais no Blender.

Técnicas discretas

- Geração de texturas -



Figura 10. Aplicação texturas procedimentais a um avião.

Técnicas discretas

- Mapeamento de texturas -

- As texturas são padrões de bits, ou bitmaps, organizados normalmente como *arrays* rectangulares de cores, que são mapeados em superfícies a desenhar.
- As texturas permitem distinguir objectos geometricamente iguais, ou seja, com tamanho e forma idênticas.

Técnicas discretas

- Mapeamento de texturas -

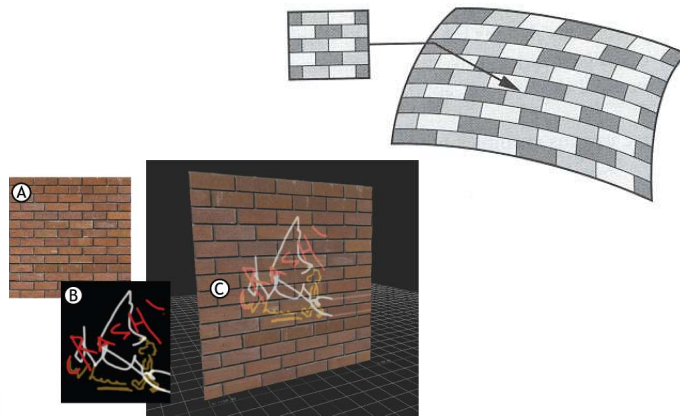


Figura 11. Mapeamento de texturas.

Técnicas discretas

- Mapeamento de texturas -

- Um mapa de textura associa um ponto da função T a cada ponto do objecto geométrico.
- A função da textura, $T(s, t)$, que descreve um qualquer padrão, é armazenada como uma matriz $m \times n$ de elementos, cujo nome é *texels*.
 - Os valores de T , são usados para atribuir/modificar propriedades da superfície, como cores ou geometria.

Técnicas discretas

- Mapeamento de texturas -

- O processo de mapeamento deve assumir que o padrão da imagem se repete fora do intervalo $T(s, t)$, conforme se vê na figura 12.



Figura 12. Mapeamento de texturas.

Técnicas discretas

- Mapeamento de texturas -

- Se o objecto geométrico é definido por uma superfície paramétrica, há que ter em conta os mapeamentos:
 - de coordenadas paramétricas para coordenadas geométricas;
 - de coordenadas de textura para coordenadas geométricas.
- Ao resultado obtido, aplica-se um novo mapeamento para obter as coordenadas do ecrã (figura 13).

Técnicas discretas

- Mapeamento de texturas -

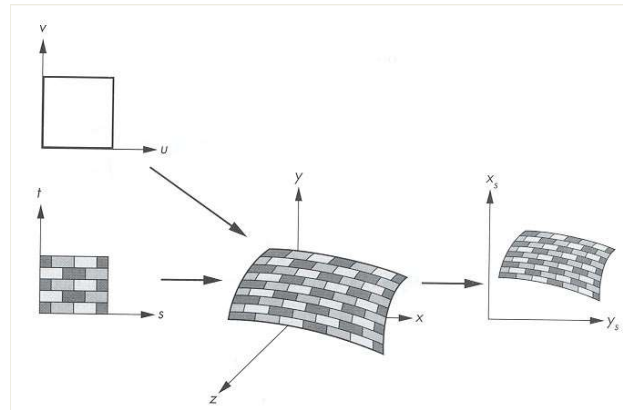


Figura 13. Mapas de textura para uma superfície paramétrica.

Técnicas discretas

- Mapeamento de texturas -

- Os principais problemas que se colocam com o mapeamento de texturas são (figura 14):
 - O mapeamento ser baseado no formato rectangular, criando para as regiões com forma arbitrária algumas dificuldades;
 - Como se pretende habitualmente mapear áreas em áreas e não pontos em pontos, existe a possibilidade de ocorrência de problemas de aliasing.

Técnicas discretas

- Mapeamento de texturas -

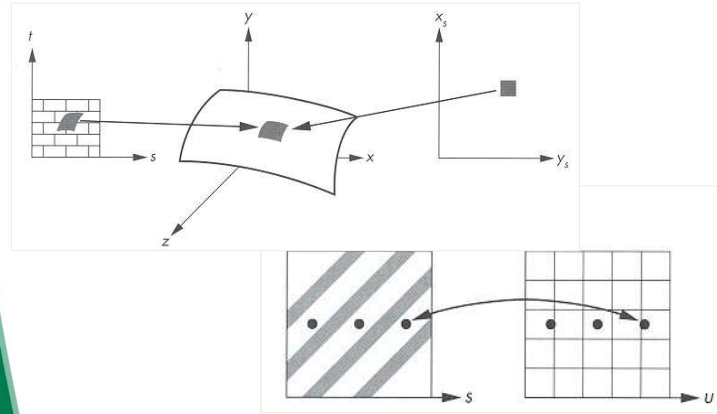


Figura 14. Principais problemas que se colocam com o mapeamento de texturas.

Técnicas discretas

- Mapeamento de texturas -

- As respectivas soluções para esses problemas podem passar pela aplicação de técnicas de:
 - UV mapping.
 - Planifica-se o objecto a mapear (ou desdobra-se para ficar planar);
 - Define quais as zonas do objecto e qual a parte da textura que lhe corresponderá;
 - Aplica-se o mapeamento a cada parte.
 - Anti-aliasing.
 - Explicado anteriormente.

Técnicas discretas

- Mapeamento de texturas -

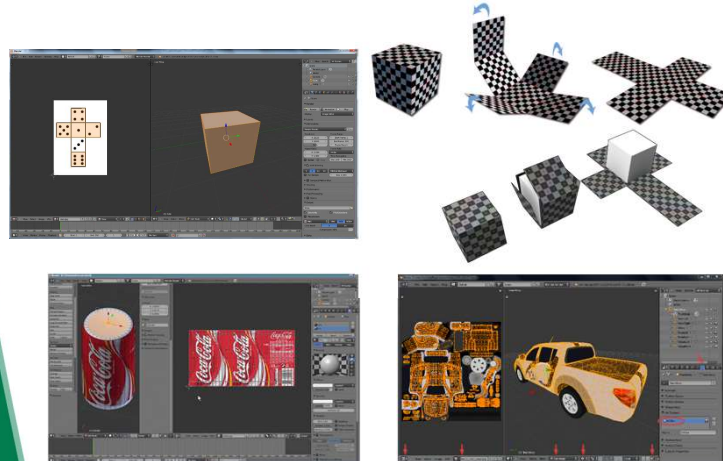


Figura 15. Exemplos de *UV mapping* feito no Blender.

Técnicas discretas

- Mapeamento de texturas -

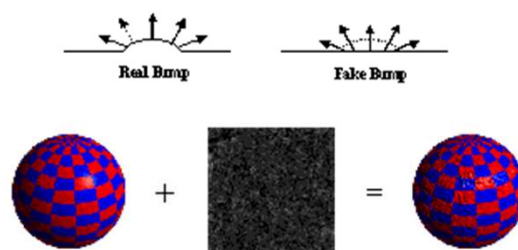


Figura 16. Mapeamento *Bump*. A superfície não muda. É o sombreamento que faz com que pareça mudada. O bump map causa deslocamentos acima e abaixo da superfície.

Técnicas discretas

- Nevoeiro -

- Nas imagens em computador, a definição é constante ao longo da cena no sentido da profundidade, isto é, a profundidade de campo é infinita dentro do volume de visualização.
- O aspecto da cena pode tornar-se pouco realista devido a uma definição exagerada para pontos mais distantes.

Técnicas discretas

- Nevoeiro -

- Acrescentando o efeito de nevoeiro (ou fog), os objectos na cena vão-se desvanecendo com a distância, como se pode verificar na figura 17.
 - Com este efeito ligado, os objectos colocados mais longe do centro de projecção começam a desvanecer para a cor do nevoeiro;
 - Serve para, além de nevoeiro, simular fumo, poluição ou qualquer efeito de limitação de visibilidade.

Técnicas discretas

- Nevoeiro -

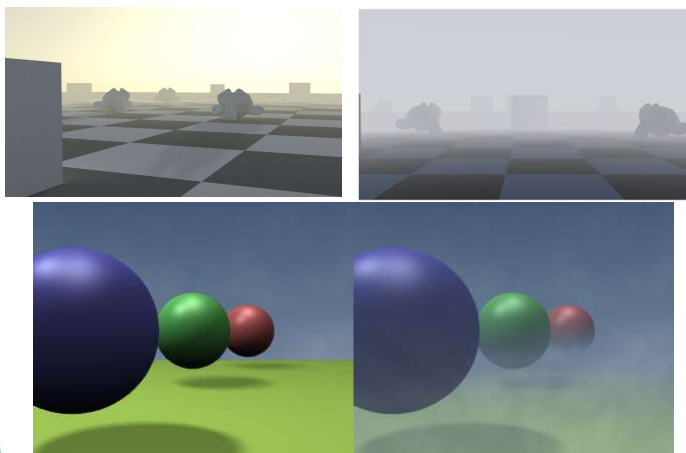


Figura 17. Imagens resultantes da aplicação do nevoeiro.

Técnicas discretas

- Nevoeiro -

- O efeito de nevoeiro é aplicado após
 - as transformações geométricas;
 - os cálculos de iluminação;
 - a aplicação de texturas.
- Assim, afecta os objectos que já sofreram esses efeitos.

Técnicas discretas

- Nevoeiro -

- O Blender permite lidar com a introdução de nevoeiro.
 - Para isso tem que se usar o painel *Mist*, da janela de propriedades, quando o ícone *World* estiver seleccionado.



Figura 18. Técnicas de nevoeiro no Blender.

Técnicas discretas

- Operações sobre fragmentos -

- As imagens são mostradas em ecrãs.
 - O ecrã é composto de um *array* rectangular de *pixels*, cada um deles com a capacidade de fazer o *display* de um pequeno quadrado de cor.
- Depois das etapas da renderização, texturização e aplicação de nevoeiro, os dados ainda não estão transformados em *pixels*, são fragmentos.

Técnicas discretas

- Operações sobre fragmentos -

- Cada fragmento tem coordenadas correspondentes a um *pixel*, assim como valores de cor e profundidade.
- Só depois do fragmento passar por certas operações é que se pode tornar num *pixel*.
 - Para desenhar um *pixel* é necessário obter informações sobre ele, e cada tipo de informação é armazenada num determinado *buffer*.

Técnicas discretas

- Operações sobre fragmentos -

- Quando se determina que um fragmento individual deverá ser gerado e que cor deverá ter, restam algumas etapas de processamento que controlam como e se ele é desenhado como um *pixel*.
 - Se, por exemplo, estiver fora do *viewport* definido ou estiver mais longe da câmara do que o *pixel* que já está no *frame buffer*, esse fragmento não será desenhado.

Técnicas discretas

- Operações sobre fragmentos -

- Há alguns testes que o fragmento terá de passar antes de ir para o *frame buffer* e possíveis operações que devem ser executadas no fragmento quando está a ser escrito.
- Se um fragmento é eliminado num determinado teste, nenhum dos testes ou operações seguintes terão lugar.

Técnicas discretas

- Operações sobre fragmentos -

- Esses testes e operações ocorrem pela ordem seguinte:
 - Scissor test;
 - Alpha test;
 - Stencil test;
 - Depth test;
 - Blending;
 - Dithering;
 - Logical operation.

Os *buffers* e a CG

- Os API's gráficos utilizam diferentes espaços de memória para guardar informação referente às imagens.
 - Esses espaços são conhecidos como *buffers*;
 - Contêm diferentes quantidades de dados por *pixel*, mas cada buffer tem a mesma quantidade de informação para cada *pixel*.
 - Cada *buffer* contém dados para realizar tarefas específicas.

Os *buffers* e a CG

- Os *buffers* são:
 - Discretos;
 - Têm uma resolução espacial, medida em *mxn pixels*;
 - Têm uma resolução de profundidade, medida em *bits/pixel*, que é dada pelo número de planos de bits, onde cada plano tem *mxn pixels*;

Os *buffers* e a CG

- Cada *pixel* é constituído por todos os k elementos de cada plano, na mesma posição espacial, podendo ser de um *bit*, um *byte*, um inteiro ou um real.

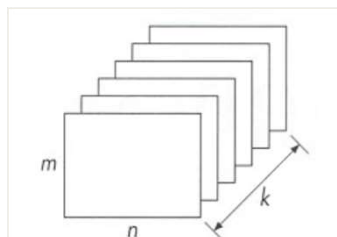


Figura 19. Representação de um buffer.

Os *buffers* e a CG

- O *buffer* que todos os API gráficos usam é o da cor (pelo menos um).
 - É também conhecido como *frame buffer*.
 - Podem conter valores de cor indexada, cor RGB ou cor RGBA.
 - Para implementações de aplicações que suportem uma visualização estereoscópica, os API's usam dois *buffers* de cor para armazenar cada uma das imagens do par estéreo.

Os *buffers* e a CG

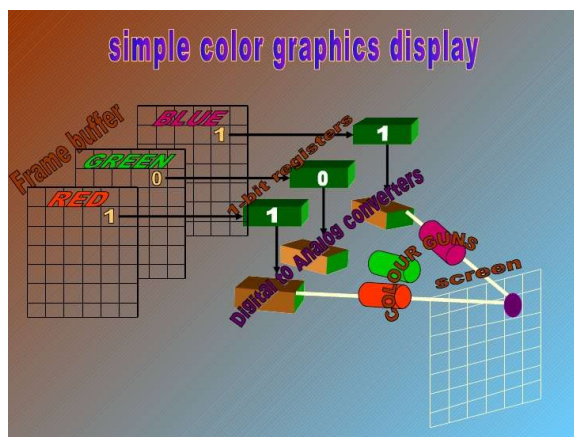


Figura 20. Funcionamento do *frame buffer*.

Os *buffers* e a CG

- Similarmente, sistemas *double-buffered* têm o *buffer* da frente e o *buffer* de trás.
- Qualquer implementação gráfica tem de usar pelo menos o *buffer* esquerdo frontal.
- Opcionalmente, podem ser suportados *buffers* de cor auxiliares não visualizáveis, sem qualquer utilização particular.

Os *buffers* e a CG

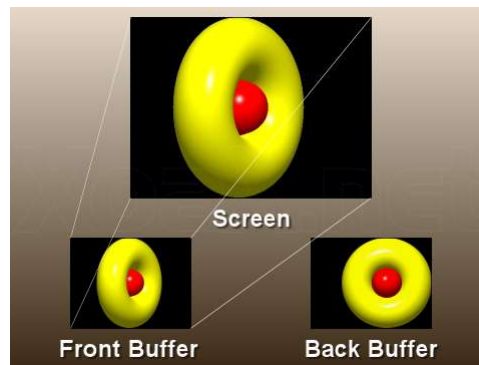


Figura 21. Funcionamento do *double buffer*.

Os *buffers* e a CG

- Por outro lado, os API's gráficos usam um outro *buffer* muito importante, o *Z-buffer* ou o *buffer* de profundidade.
 - Armazena um valor de profundidade para cada *pixel*.
 - A profundidade é medida usualmente em termos de distância dos objectos ao centro de projecção.
 - Este *buffer* é utilizado para remoção de superfícies ocultas.

Os *buffers* e a CG

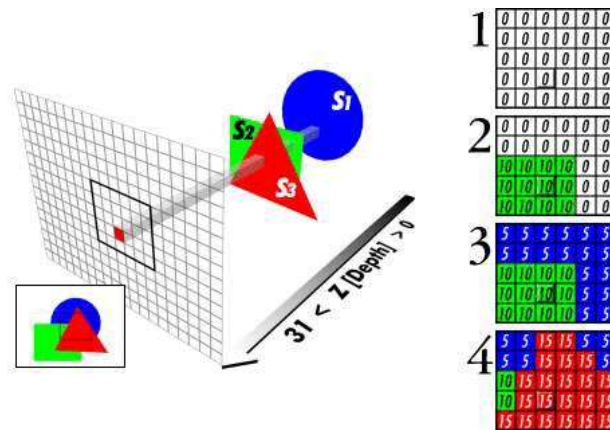


Figura 22. Funcionamento do Z-buffer.

Os *buffers* e a CG

- Dos *buffers* mais conhecidos e usados na área da CG, resta falar do *buffer de stencil*.
- Este *buffer de stencil* é utilizado para restringir o desenho a certas zonas.
- Por isso é usado principalmente em operações de mascaramento.

Os *buffers* e a CG

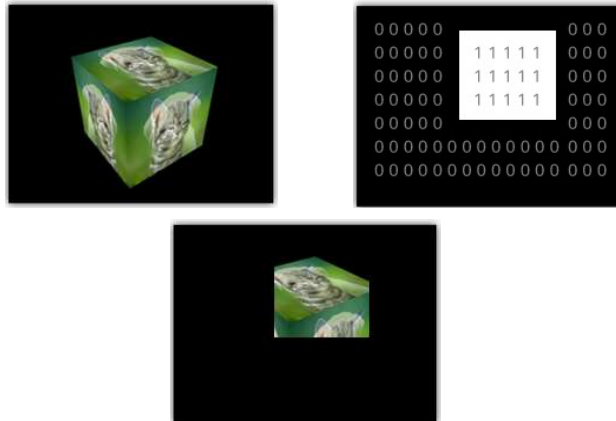


Figura 23. Funcionamento do buffer de stencil.